QuillAudits

AUDIT REPORT

February 2026

For

tria

Table of Content

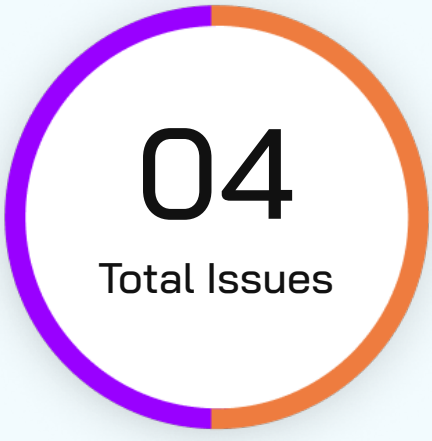
Executive Summary

Project Name	Tria
Protocol Type	Staking, Escrow, Airdrop
Project URL	https://www.tria.so/en
Overview	The TriaStakingPool functions as a fixed-APR savings vault where users lock tokens for a set duration to earn time-based rewards, The TriaSpend contract acts as a simple escrow wallet where whitelisted depositors (like the airdrop contract) credit user balances for later withdrawal, but it is currently incompatible with fee-on-transfer tokens due to strict equality checks. Finally, TriaClaim serves as the distribution engine, allowing users to claim airdropped tokens via Merkle proofs and routing them to either the Staking Pool (fee-free) or the Spend contract (charging an ETH fee), effectively incentivizing users to lock their funds rather than withdrawing them immediately.
Audit Scope	The scope of this Audit was to analyze the Tria Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/TriaHQ/tria-core-contracts/tree/main/contracts
Contracts in Scope	TriaClaim.sol, TriaSpend.sol, TriaStakingPool.sol
Commit Hash	895f75b0388b007cb8a5f2640bdb5f972723b4db
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	31st January 2026
Updated Code Received	31st January 2026
Review 2	2nd February 2026
Fixed In	a65a4b00f5d2315383a39d2e0d05f275e3106a3c

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0(0.0%)
High	0(0.0%)
Medium	2 (50.0%)
Low	0(0.0%)
Informational	2 (50.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	**2**	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	0	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Permanent “Lifetime” Deposit Lockout (DoS)	Medium	Acknowledged
2	Emergency Withdraw is Blocked by Pause	Medium	Acknowledged
3	Override Mutability Should Match Parent for EIP Compliance	Informational	Resolved
4	Token Address Mutability May Break Accounting Assumptions	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Medium Severity Issues

Potential Gas-Griefing via Unbounded Deposit Storage

Acknowledged

Path

TriaStakingPool.sol

Function Name

`deposit()`

Description

The deposit() function allows deposits to be made for an arbitrary _recipient, and user deposits are stored in an append-only array. Although deposits can be withdrawn, withdrawn entries are only zeroed out and not removed from storage, causing unbounded growth of the per-user deposits array over time.

While removal of maxDepositsPerUser eliminates the permanent lockout vector, a malicious actor can still intentionally create many Minimum value deposits for a victim address, inflating storage and increasing gas costs for withdrawal operations.

Impact

Increased gas costs for withdrawals Degraded user experience under adversarial conditions

Likelihood

Medium

Recommendation

Implement storage cleanup or indexing (e.g., head/tail pointers) to skip withdrawn entries

Tria Team's Comment

Right now there is no storage clearing. We designed this so that we would have the state of deposits, but based on this and the attack, we will surely add storage cleanup. We are avoiding to this at such a late stage to avoid any issues but we will surely implement this as the contract is upgradeable. Since our withdrawal would be live after a year, we would have time to review this functionality in a much better way and thus fix this.

There is a minDeposit of 10$ so this makes griefing quite costly but the storage clearing would still be implemented soon.

Emergency Withdraw is Blocked by Pause

Acknowledged

Path

TriaStakingPool.sol

Function Name

`emergencyWithdraw()`

Description

The emergencyWithdraw function is intended as a safety hatch for users to retrieve their principal in case of critical issues. However, it includes the whenNotPaused modifier. If the Admin detects a bug and pauses the contract to stop exploits, they inadvertently trap all honest users, preventing them from using the emergency exit.

Impact

User funds can be frozen indefinitely if the contract is paused. Users lose their only trustless exit mechanism during emergencies.

Likelihood

Medium (Specific to emergency scenarios, but critical impact).

Likelihood

1. Admin calls pause().
2. User calls emergencyWithdraw(userAddress).
3. Transaction reverts due to Pausable: paused.

Recommendation

Remove the whenNotPaused modifier from the emergencyWithdraw function. It should be callable even (and especially) when the contract is paused.

Tria Team's Comment

This is intentional. We have disabled emergency withdraw during deployment. We just want to keep this functionality open if needed for users to remove their funds. If this is enabled, the overall contract won't be paused. Deposits and withdrawals can still be micro managed due to the respective flags so technically we can enable the emergency withdraw and not pause the system. When I meant this can stay on is that this can be enabled in normal systems where everything is working (unpaused state) Let me know if this makes sense.

Informational Issues

Override Mutability Should Match Parent for EIP Compliance

Resolved

Path

TriaStakingPool.sol, TriaSpend.sol, TriaClaim.sol

Function

`_authorizeUpgrade`

Description

An overridden function does not strictly match the mutability of its parent implementation. While this does not affect runtime behavior in the current context (as the function is intentionally empty and upgradeToAndCall is used), mismatched mutability can break interface expectations and deviate from EIP and OpenZeppelin override best practices.

Impact

Potential deviation from EIP/interface compliance

Likelihood

Low

Recommendation

Update the overridden function to exactly match the parent mutability to maintain EIP compliance and long-term maintainability.

Token Address Mutability May Break Accounting Assumptions

Resolved

Path

TriaSpend.sol

Function

`withdraw`

Description

The contract strictly enforces standard ERC20 behavior (including fee-on-transfer protection), which is good. However, the ability to update the token address after deployment introduces a scenario where changing the token between deposits and withdrawals could invalidate accounting assumptions (e.g., balances and totalDeposited no longer mapping to the same asset).

Impact

No immediate vulnerability but Potential accounting inconsistency if misused.

Likelihood

Low

Recommendation

Improve accounting flexibility to better handle non-standard ERC20 transfer behavior during withdrawals.

Tria Team's Comment

We have removed the update function that had the functionality to update the token address in Spend contract.

Functional Tests

Some of the tests performed are mentioned below:

- **Should allow a user to deposit tokens and record the unlockTime based on the configuration.**
- **Should revert if the user attempts to withdraw before the minLockDuration has passed.**
- **Should allow withdrawal of principal + rewards once the lock duration and globalUnlockTimestamp have passed.**
- **Should allow withdrawing half of a deposit and calculate rewards only on that half.**
- **Should increase the recipient's balance when a whitelisted address calls depositFor.**
- **Should allow a user to withdraw their full balance if the contract holds sufficient tokens.**
- **Should successfully process a claim if the user provides a valid Merkle proof and amount.**
- **Should revert with AlreadyClaimed if the user attempts to claim a second time.**
- **Should revert the claim if the TriaClaim contract is not whitelisted on TriaSpend (when allocating to Spend).**

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
OpenZeppelin Contracts	Library	Core logic for Ownable, Pausable, ReentrancyGuard, SafeERC20	Correct implementation of standard patterns	N/A
ERC20 Token	External Contract	The asset used for Staking, Spending, and Claiming	Complies with standard ERC20 behavior	Fee-on-transfer, rebasing, or malicious implementations can cause insolvency
MerkleProof	Tool	Compilation	Compiler has no known critical bugs	Older version can lead to unsafe defaults
Fee Recipient	External Address	Receives ETH fees from TriaClaim	Address is capable of receiving ETH	DoS Risk: If set to a contract without receive(), all claims will revert.
TriaStakingPool	Internal Dependency	TriaClaim deposits directly into this contract	Interface matches and limits are respected	N/A

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
TriaSpend	Internal Dependency	TriaClaim deposits directly into this contract	Whitelist is configured correctly	Critical Config: TriaClaim must be whitelisted on TriaSpend or claims fail.

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
TriaStakingPool.sol	initialize	**What this function can do** Set initial APR, caps, timestamps, owner **What this function cannot/should not do** Be called more than once **Main invariant(s)** stakingToken != address(0) **Access level** Public (once)

Contract Name	Function	Category
TriaStakingPool.sol	withdraw	**What this function can do** Return principal + rewards **What this function cannot/should not do** Withdraw before unlock (unless global unlock reached) **Main invariant(s)** totalStaked -= amount **Access level** Public
TriaStakingPool.sol	emergencyWithdraw	**What this function can do** Withdraw any token from contract **What this function cannot/should not do** N/A (trust-based) **Main invariant(s)** None **Access level** Admin
TriaSpend.sol	TriaSpend.sol	**What this function can do** Credit user balance **What this function cannot/should not do** reverts on fee-on-transfer tokens **Main invariant(s)** totalDeposited += amount **Access level** Whitelisted

Contract Name	Function	Category
TriaSpend.sol	withdraw	**What this function can do** User withdraws their balance **What this function cannot/should not do** Withdraw more than stored balance **Main invariant(s)** totalDeposited -= amount **Access level** Public
TriaClaim.sol	claim()	**What this function can do** Distribute tokens to Stake/Spend **What this function cannot/should not do** Claim twice, bypass Merkle proof **Main invariant(s)** totalClaimed += amount **Access level** Public
TriaClaim.sol	claimDirect	**What this function can do** Send tokens directly to user **What this function cannot/should not do** Bypass fees **Main invariant(s)** totalClaimed += amount **Access level** Public

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
TriaStakingPool	ERC20	User Principals + Reward Reserves
TriaSpend	ERC20	User Balances (Escrow)
TriaClaim	ERC20	Unclaimed Airdrop Supply

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
StakingPool	deposit	ERC20 (User)	-	Public	Gas Griefing Risk: Deposit Lockout bug.
StakingPool	withdraw	-	ERC20 (User)	Public	Math Risk: Reward calculation on partial withdraws.
StakingPool	fundRewards	ERC20 (Admin)	-	Owner	Safe.

Closing Summary

In this report, we have considered the security of Tria Smart Contract. We performed our audit according to the procedure described above.

Two Medium and Two Informational Issues found during the Audit, the Tria Team acknowledged few and resolved the remaining issues.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

7+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1400+** Projects Secured

Follow Our Journey

AUDIT REPORT

February 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com